End of Day Report

Name	Atibali Saiyed
Date	05 November 2026
Repository	https://github.com/Atibali/Gateway-Training/tree/main/Week1/Day3

Part 1 — Object-Oriented Programming in C#

Classes and objects — Learned that a class is a blueprint and an object is an instance created using `new`. Covered fields for storing data; properties using `get`, `set`, and `init` for controlled access; auto-properties; constructors for initialization; `this` to refer to the current instance; and object initializers for concise object creation.

Encapsulation — Learned to protect internal state using access modifiers such as `public`, `private`, `protected`, and `internal`, while exposing controlled access through properties. Read-only properties help protect data, and property setters can validate values and throw exceptions when invalid input is provided.

Inheritance — Understood how derived classes extend base classes using `:` to reuse and specialize behaviour. Practised using `base` to access base-class members, constructor chaining with `: base(...)`, and the `sealed` keyword to prevent a class from being inherited further.

Polymorphism — Learned that polymorphism allows the same operation to take different forms. Method overloading provides compile-time polymorphism through different method signatures. `virtual` and `override` enable runtime polymorphism through dynamic dispatch. Also learned that `new` hides a base-class member; the selected member depends on the reference type rather than runtime polymorphic dispatch.

Abstraction — Learned to hide implementation complexity behind abstract classes and interfaces. Abstract classes can contain state, shared implementation, and abstract methods that derived classes must implement. Interfaces define contracts, and a class can implement multiple interfaces even though C# supports single class inheritance.

Static members — Understood that static members belong to the type rather than to individual object instances. Static fields are shared across instances, static methods are called through the class itself, and static classes are useful for grouping utility functionality and cannot be instantiated.

Structs and records — Compared structs, which are value types copied on assignment, with classes, which are reference types whose references can point to the same object. Learned that records provide value-based equality, non-destructive mutation through `with` expressions, and a generated `ToString` method, making them useful for representing data.

Composition over inheritance — Learned to prefer a `has-a` relationship through composition over an `is-a` relationship through inheritance unless a genuine subtype relationship exists. Composition improves flexibility because behaviours can be replaced at runtime through interfaces, reducing tight coupling to a class hierarchy.

SOLID Principles

Single Responsibility Principle (SRP) — A class should have one responsibility and one reason to change.

Open/Closed Principle (OCP) — Software entities should be open for extension but closed for modification.

Liskov Substitution Principle (LSP) — Derived types should be usable in place of their base types without breaking expected behaviour.

Interface Segregation Principle (ISP) — Clients should not be forced to depend on interface methods they do not use.

Dependency Inversion Principle (DIP) — High-level and low-level modules should depend on abstractions rather than concrete implementations.

Summary

Today I learned the core concepts of object-oriented programming in C#, including classes and objects, encapsulation, inheritance, polymorphism, abstraction, static members, structs, and records. I also understood why composition is often more flexible than inheritance and was introduced to the five SOLID principles: SRP, OCP, LSP, ISP, and DIP.

Key learning outcome: Built a foundation for designing C# applications using object-oriented principles and writing code that is easier to maintain, extend, and test.